Module 5
Functional Architecture

Agenda

1. Introduction to Functions
2. Anatomy of Functions
3. Return Values
4. Scope
5. Type Hinting

© Copyright Intellipaat. All rights reserved.

Introduction to Functions

© Copyright Intellipaat. All rights reserved.

The Problem with Repeated Code

When we write the same lines of code again and again, our program becomes **difficult to maintain**.

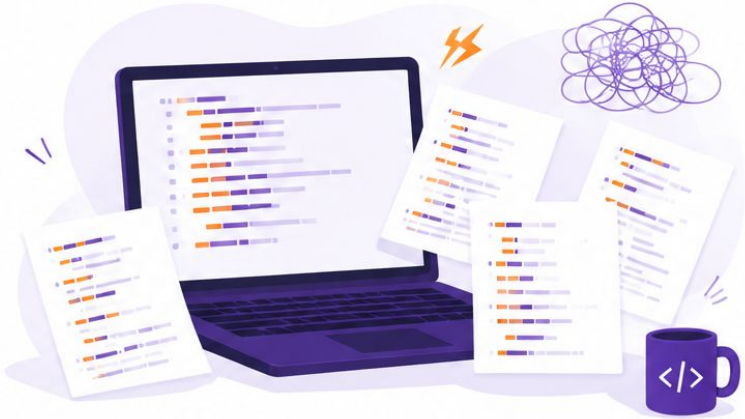
Example

```
1   print("Welcome Manan")
2   print("Welcome Ali")
3   print("Welcome Sara")
4   print("Welcome John")
5   print("Welcome Mike")
..  ...  (and many more)
```

Same logic, repeated again and again...

Think about it!

- What if we need to greet 100 people?
- What if the greeting message changes?



Repetition creates problems:

Hard to manage
Code becomes longer and harder to read and understand.

Error-prone
Fixing a mistake in one place is easy, but fixing it in many places is risky and time-consuming.

© Copyright Intellipaat. All rights reserved.

Can We Reuse Code?

Instead of writing the same code many times,
can we write it **once** and use it **multiple times?**

What we do now ✗

We repeat the same lines again and again.

```
print("Welcome Manan")
print("Welcome Ali")
print("Welcome Sara")
print("Welcome John")
print("Welcome Mike")
...
```

Longer code, more chances of errors, and difficult to maintain.

Isn't there a better way?



Yes!

We can package the logic in a block of code and **reuse** it whenever needed.

What if we could ✓

Write once, use many times!

```
def greet(name):
    print("Welcome", name)

greet("Manan")
greet("Ali")
greet("Sara")
...
```

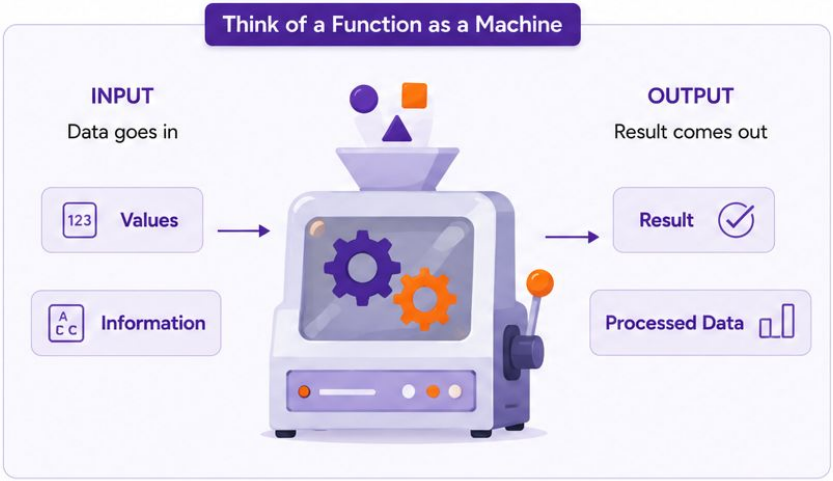
Shorter code, easy to update, and reusable everywhere!

This is the power of **Functions!** We will learn how to do this next.

© Copyright Intellipaat. All rights reserved.

What is a Function?

A function is a **reusable block of code** that performs a **specific task**.



Why Functions?

Reusable
Write once, use many times.

Organized
Break complex problems into smaller, manageable parts.

Reliable
Easier to test, debug, and maintain.

Efficient
Less code, better readability, more productivity.

Functions help us write better, cleaner, and smarter Python programs.

© Copyright Intellipaat. All rights reserved.

DRY Principle

DRY (Don't Repeat Yourself)

DRY stands for **Don't Repeat Yourself**.
It is a key principle for writing clean, reusable, and maintainable code.

Not DRY

Same logic written multiple times

```
print("Welcome Manan")
print("Welcome Ali")
print("Welcome Sara")
print("Welcome John")
print("Welcome Mike")
⋮
```

Hard to modify, easy to make mistakes.

DRY

Write once, use many times

```
def greet(name):
    print("Welcome", name)

greet("Manan")
greet("Ali")
greet("Sara")
greet("John")
greet("Mike")
⋮
```

Easy to update, reusable, and maintainable.

Benefits of DRY

Write Once
Create the logic in one place.

Easy to Maintain
Change the code once, and the update applies everywhere.

Save Time & Effort
Reuse existing code instead of rewriting it.

Remember: Good programmers don't write more code,
They write **better code**.

© Copyright Intellipaat. All rights reserved.

Hands-On: Introduction to Functions

© Copyright Intellipaat. All rights reserved.

Hands-On: Introduction to Functions

Problem Statement

A fitness application needs to remind users to stretch during work hours.

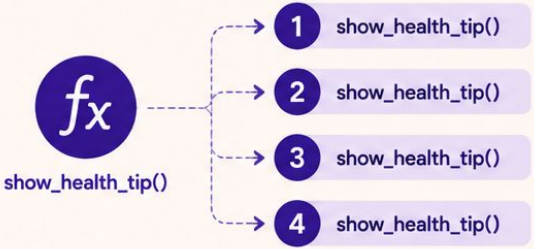
- Instead of writing the same **print()** statement repeatedly, create a function named **show_health_tip**.
- The function should display the message **Take a short stretch break.**
- Call the function **4** times.

Goal:
Use a function to avoid repeating code and promote healthy work habits!



Expected Output

```
Take a short stretch break.
Take a short stretch break.
Take a short stretch break.
Take a short stretch break.
```





© Copyright Intellipaat. All rights reserved.

Anatomy of Functions

© Copyright Intellipaat. All rights reserved.

Anatomy of a Function

Every function in Python has these **key parts**.

1 Keyword (def)
Tells Python that we are defining a function.

2 Function Name
The name you choose to identify the function.

3 Parameter
Input that the function accepts (optional).

```
1  def greet(name):
2      message = "Welcome, " + name
3      return message
4
5  greet("Manan")
```

4 Function Body
The block of code that performs the task.

5 Return Statement
Sends a result back to the caller (optional).

6 Function Call
The function is called to execute the code.

Think of a function like a **mini-machine:**
Inputs go in (**parameters**), processing happens (**body**), and output comes out (**return**).



Function Lifecycle



1. Input
Values are passed to the function (parameters).



2. Processing
The function processes the input using the code inside its body.



3. Output
The result is returned (if any) to the caller.

© Copyright Intellipaat. All rights reserved.

Function Naming Rules

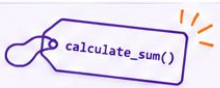
Follow these simple rules to write **clear, readable,** and **professional** function names in Python.

1. **Use lowercase letters**
   Use only lowercase letters to keep it consistent.
2. **Use underscores**
   Separate words with underscores.
3. **Don't start with numbers**
   Function names cannot start with a number.
4. **Use only letters, numbers, and underscores**
   Avoid special characters like !, @, #, etc.
5. **Be meaningful**
   Choose names that describe what the function does.

Good vs Avoid Examples

Good		Avoid
`calculate_sum()`	VS	`CalculateSum()`
`get_user_name()`	VS	`getUserName()`
`get_data()`	VS	`1get_data()`
`print_result()`	VS	`print-result()`
`calculate_average()`	VS	`func()`

Remember: A good function name is like a good label—it tells you exactly what's inside!



© Copyright Intellipaat. All rights reserved.

Function Parameters

Functions can accept input using **parameters**.
Let's understand the basics.

What is a Parameter?

A **parameter** is a variable listed inside the parentheses ( ) in the function definition.

What is an Argument?

An **argument** is the actual value that we pass to the function when we call it.

How it works



1. You pass a value **(Argument)**
2. Function receives it **(Parameter)**
3. Function processes it and returns result

Example

```
# Function Definition
def greet(name):
    print("Hello, " + name + "!")

# Function Call
greet("Manan")
```

Parameter
Placeholder name

Argument
Actual value "Manan"

In simple words:

Parameters are like **placeholders** in the function.
Arguments are the **real values** you pass.



© Copyright Intellipaat. All rights reserved.

Why Do We Need Parameters

Without parameters, functions can only work with **fixed values**.
With parameters, the same function can handle **different inputs**.

Without Parameters

Works only for **one specific value**.

```
def greet():
    print("Hello, Manan!")
```

```
greet()     # Output: Hello, Manan!
```

✗ Cannot greet someone else without changing the code.

Problem:
To greet another person, we must rewrite the function.

Parameters make functions **flexible** and **reusable**.

With Parameters

Works for **many different values**.

```
def greet(name):
    print("Hello, " + name + "!")
```

```
greet("Manan")  → Hello, Manan!
greet("Sara")   → Hello, Sara!
greet("Arjun")  → Hello, Arjun!
```

✓ The same function works for everyone!



Key Takeaway

Parameters let us write functions **once** and use them many times with **different inputs**.

© Copyright Intellipaat. All rights reserved.

Positional Arguments

In positional arguments, values are passed to a function in the **same order** as the parameters are **defined**.

How it works

1. You pass values in order
2. Function receives them in the same order
3. Function uses them accordingly



Values are mapped to parameters based on their **position**.

First value → First parameter

Second value → Second parameter

Example

```
# Function definition
def add(a, b):
    return a + b

# Function calls (positional order)
add(3, 5)      # 3 → a, 5 → b      Output: 8
add(10, 2)     # 10 → a, 2 → b     Output: 12
add(-4, 7)     # -4 → a, 7 → b     Output: 3
```

Parameters
a, b

Visual Mapping

`add(3, 5)`
Arguments
(passed in order)

`def add(a, b):`
Parameters
(defined in order)

Key Takeaway: In positional arguments, **order matters!**
Always pass values in the correct sequence.

© Copyright Intellipaat. All rights reserved.

Default Parameters

Default parameters have a **default value**.
If no value is provided during the function call, the **default value** is used.

What are Default Parameters?

Parameters in a function can have a **default value**.
If we do not pass any value, the **default** is used automatically.

How it works



1. Function is called with or without values
2. Python checks which values are provided
3. Missing values are filled with **default values**

Example

```
# Function definition
def greet(name, msg="Hello!"):
    print(f"{msg} {name}!")

# Function calls
greet("Manan")          # Only name is provided
greet("Sara", "Hi!")    # Both name and msg are provided
greet("Arjun")          # msg uses default value
```

Default Parameter
`msg = "Hello!"`

Output Hello! Manan!

Output Hi! Sara!

Output Hello! Arjun!

Note:
Default parameters must come after non-default parameters.

```
def func(a=10, b):
# ✗ Invalid
```

```
def func(a, b=10):
# ✓ Valid
```

Key Takeaway: Default parameters make functions **flexible** and **easier** to use by providing **backup values**.

© Copyright Intellipaat. All rights reserved.

Variable-Length Parameters(*args)

Sometimes, we don't know how many values a function will receive.
*args allows a function to accept **any number of positional arguments**.

What is *args?

*args is a special parameter that collects all extra positional arguments into a **tuple**.

How it works



1. You pass any number of arguments.
2. Python packs them into a **tuple**.
3. The function uses them as needed.

Note:
*args must be the **last** parameter in the function definition.

Example

```
# Function definition
def total(*args):
    print("Numbers received:", args)
    print("Sum =", sum(args))
```

*args collects all values into a **tuple** named **'args'**

Function Calls

```
total(1, 2, 3)          # 3 arguments
```

Output:
```
Numbers received: (1, 2, 3)
Sum = 6
```

```
total(10, 20, 30, 40, 50)  # 5 arguments
```

Output:
```
Numbers received: (10, 20, 30, 40, 50)
Sum = 150
```

```
total()                 # no arguments
```

Output:
```
Numbers received: ()
Sum = 0
```

Behind the scenes

Call `total(1, 2, 3, 4)` → *args collects all values → args = (1, 2, 3, 4) (tuple) → Function uses the tuple

Key Takeaway: Use *args when you are **not sure how many values** the function will receive.
It makes your functions **flexible** and **reusable**.

© Copyright Intellipaat. All rights reserved.

Keyword Arguments

In keyword arguments, values are passed using the parameter names. So, the order doesn't matter.

Why use Keyword Arguments?

- Makes code more readable
- Order of values doesn't matter
- Each value goes to the right parameter

Example

Parameters
name, age, city

```
# Function definition
def introduce(name, age, city):
    print(f"Hello {name}, you are {age} years old and live in {city}.")
```

1

```
# Call 1: Any order
introduce(name="Manan", age=20, city="Pune")
```

name age city

Output
Hello Manan,
you are 20 years old
and live in Pune.

2

```
# Call 2: Different order
introduce(city="Pune", age=20, name="Manan")
```

name age city

Output
Hello Manan,
you are 20 years old
and live in Pune.

Key Takeaway: Use keyword arguments when you want clarity and flexibility.

© Copyright Intellipaat. All rights reserved.

Hands-On: Anatomy of Functions

© Copyright Intellipaat. All rights reserved.

Hands-On: Anatomy of Functions

Problem Statement

- Create a clearly named function called **display_workshop**.
- It should accept the **workshop name** and **instructor** as parameters.
- It should also have a default parameter named **venue** with the value **Auditorium**.
- Call the function once using only **positional arguments** so that the default venue is used.
- Call it a second time by providing the venue as a **keyword argument**.
- Display the **workshop name**, **instructor**, and **venue** for both sessions.

What the Function Should Do

Workshop Details In → **display_workshop** Function → Workshop Details Displayed

Function Parameters

- **workshop_name** — Name of the workshop
- **instructor** — Name of the instructor
- **venue (default)** — Venue of the workshop (default: **Auditorium**)

Expected Output



1 Using Positional Arguments (Default Venue Used)

Workshop: **Python Basics**
Instructor: **Priya**
Venue: **Auditorium**

2 Using Keyword Argument (Custom Venue Provided)

Workshop: **Cloud Computing**
Instructor: **Rahul**
Venue: **Seminar Hall**



© Copyright Intellipaat. All rights reserved.

Return Values

© Copyright Intellipaat. All rights reserved.

Functions Can Return Values

So far, we used functions to perform tasks. But functions can **also return values** that can be used elsewhere in the program.

What is a Return Value?

A **return value** is the result that a function sends back to the place where it was called.

Easy Analogy



You give **ingredients** to the cook. → Cook processes them. → Cook **returns** the final dish.

Similarly, a function **processes** the input and **returns** a value.

Example

```
# Function definition
def add(a, b):
    result = a + b
    return result

# Function call
sum_value = add(4, 6)
print(sum_value)
```

Function returns the **result**

Returned value is stored in **sum_value**

Output

```
10
```

How it works



1. Function is called
2. Function does its work
3. Function returns a value
4. Value is used in the program

Key Takeaway: Functions can return values using the **return** keyword. These values can be stored, printed, or used in further calculations.

© Copyright Intellipaat. All rights reserved.

print() vs return

Both are used inside functions, but they behave very diferently.

Using print ()

- Displays output on the screen.
- The value is **NOT returned** to the caller.

```
def greet(name):
    print(f"Hello, {name}!")

# Function call
greet("Manan")

# What happens?
Hello, Manan!
```

Output appears on the **screen**.
Nothing is stored to be used later.

VS

Using return

- Sends a value back to the caller.
- The value is **returned** and can be stored or used.

```
def greet(name):
    return f"Hello, {name}!"

# Function call
message = greet("Manan")

# What happens?
message now contains: Hello, Manan!
```

Value is **returned** to the caller.
It can be stored in a variable or used in expressions.

© Copyright Intellipaat. All rights reserved.

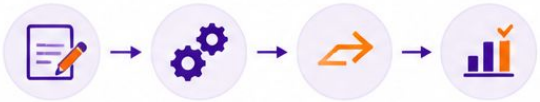
Example: Using return

Let's see how a function can return a value and how we can **use that value** in our program.

Goal

Write a function that calculates the area of a rectangle and **returns** the result.

How it works



1. Call the function with length and width.
2. Function calculates **area.**
3. Function **returns** the **area.**
4. Returned value is stored **and used.**

Example Code

```
# Function definition
def area(length, width):
    result = length * width
    return result

# Function call
l = 10
w = 5
rect_area = area(l, w)

# Using the returned value
print(f"Area of rectangle: {rect_area}")
```

Function calculates area and **returns** it to the caller

Returned value is stored in **rect_area**

We can now use **rect_area** in our program

Output

```
Area of rectangle: 50
```

Note: The value returned by a function can be used in calculations, assigned to variables, passed to other functions, etc.

Without **return**, the function would only print the result but we wouldn't be able to **reuse** it.

© Copyright Intellipaat. All rights reserved.

Side Effects

Not all functions return values.
Some functions change something outside the function.



What is a Side Effect?

A side effect occurs when a function modifies something outside its own scope, instead of (or in addition to) returning a value.

Common Side Effects



Changing global variables
Alters values that exist outside the function.



Modifying mutable objects
Changes the contents of lists, dictionaries, etc.



I/O Operations
Printing to screen, writing to files, sending emails, etc.



Example

This function changes a global variable outside its scope.

```
count = 0

def increment():
    global count
    count += 1    # modifies global variable

increment()       # function call
print(count)      # Output: 1
```



The function did not return a value, but it changed the data outside itself.



Tip: Side effects can be useful in some situations, but too many side effects can make code harder to understand and debug.



© Copyright Intellipaat. All rights reserved.

Hands-On: Return Values

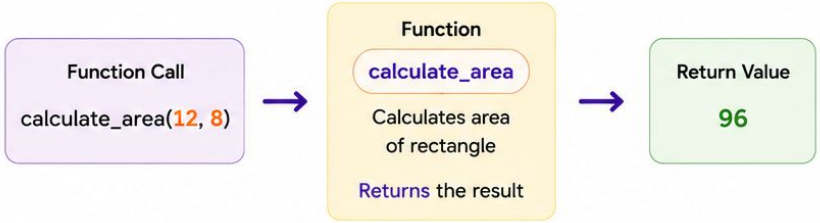
© Copyright Intellipaat. All rights reserved.

Hands-On: Return Values

Problem Statement

- Create a function named **calculate_area** that receives the length and width of a rectangle.
- The function must calculate the area and **return** the result to the function call.
- **Do not** print the result directly inside the function; store the returned value in a variable and print it **outside** the function.
- Use a length of **12** and a width of **8**.

What is a Return Value?



The function calculates the area and sends (**returns**) the result back to where it was called.

Expected Output

Output on Screen

Length: 12
Width: 8
Area: 96

How It Works



1. **Input Values**
   Length = 12
   Width = 8
2. **Function Processes**
   Area = Length × Width
   = 12 × 8
   = 96
3. **Return Value**
   Function returns 96
4. **Print Outside Function**
   Stored in a variable and printed outside the function

© Copyright Intellipaat. All rights reserved.

Scope

© Copyright Intellipaat. All rights reserved.

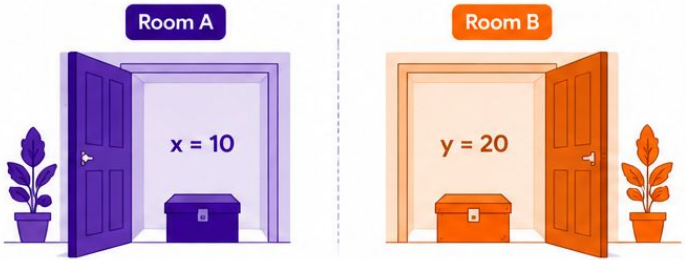
Memory Boundaries

Every **variable** lives in a place. Python respects those **boundaries**.

Core Idea

Functions create their own **memory "rooms"**.
Variables created inside a room stay there and are not visible in other rooms.

Analogy: Rooms



You can use items inside your room,
but you **cannot access** items from another room.

To share, you must **pass** the item (value) through the door.
This is how data moves between rooms.

Memory Rooms in Python

Python has different **memory areas**:

Shared Room
Stores variables available to all rooms.
Accessible everywhere.

Room A
Stores its own variables.
Visible only inside this room.

Room B
Stores its own variables.
Visible only inside this room.

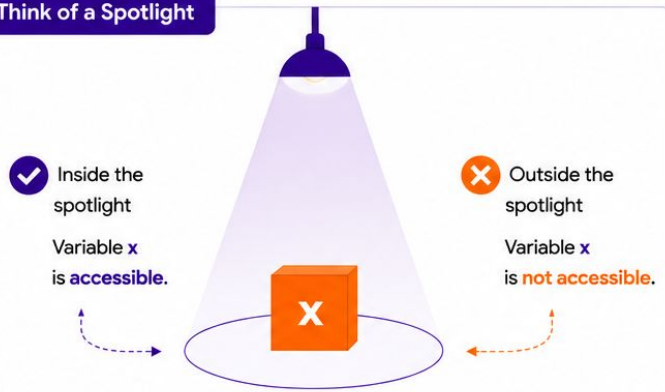
Different rooms can hold different variables with the same name.
These boundaries are called scope. We will understand scope next.

© Copyright Intellipaat. All rights reserved.

What is a Scope?

Scope is the region of a program where a variable can be accessed.

Think of a Spotlight



Scope tells us

- Exists **where?**
- Accessible **where?**
- Hidden **where?**

Simple Example

Consider the variable **x**.

```
x = 10

def demo():
    y = 20
    print(y)

demo()
```

x is created here. Its scope is the **entire program.**

y is created inside **demo()**. Its scope is **only inside demo().**

Question

Can we use **y** outside the function?
What about **x**?

Key Takeaway: Scope decides the **visibility** and **lifetime** of a variable in your program. Understanding scope helps you write correct and bug-free code.

© Copyright Intellipaat. All rights reserved.

Local Variables

Variables created inside a function belong only to that function.

What is it?



A **local variable** is created inside a function and can be used **only inside** that function. It cannot be accessed from outside the function.

Example

```
def show():
    x = 10
    print("Inside function, x =", x)

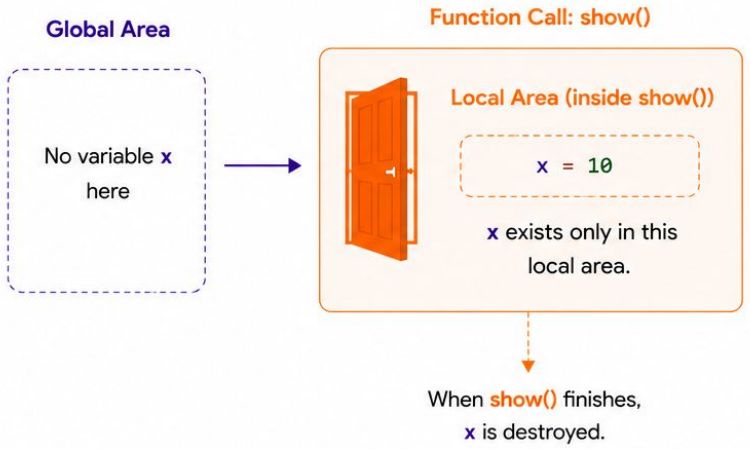
show()

# Trying to access x outside the function
print(x)
```

Output

```
Inside function, x = 10
NameError: name 'x' is not defined
```

How it Works



Remember: What happens inside a function, stays inside that function.
Local variables are **private** to their function.

© Copyright Intellipaat. All rights reserved.

Global Variables

Variables created **outside all functions** belong to the **entire program**.

What is it?

A **global variable** is created outside any function and can be used **anywhere** in the program, inside or outside functions.

Example

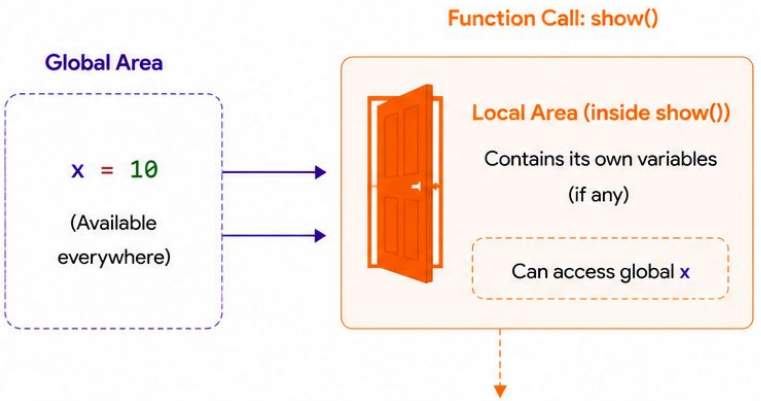
```
x = 10      # Global variable
def show():
    print("Inside function, x =", x)

show()
print("Outside function, x =", x)
```

Output

```
Inside function, x = 10
Outside function, x = 10
```

How it Works



The global variable **x** stays in the **global area** and remains avaiable throughout the program.

Remember: Global variables are shared by all functions. Use them carefully to avoid unexpected changes.

© Copyright Intellipaat. All rights reserved.

Local Shadows Global

A local variable with the same name as a global variable **shadows** it inside the function.

What is it?

If a **local variable** has the same name as a global variable, the local one is used **inside** the function.
The global one is **hidden** (shadowed).

Example

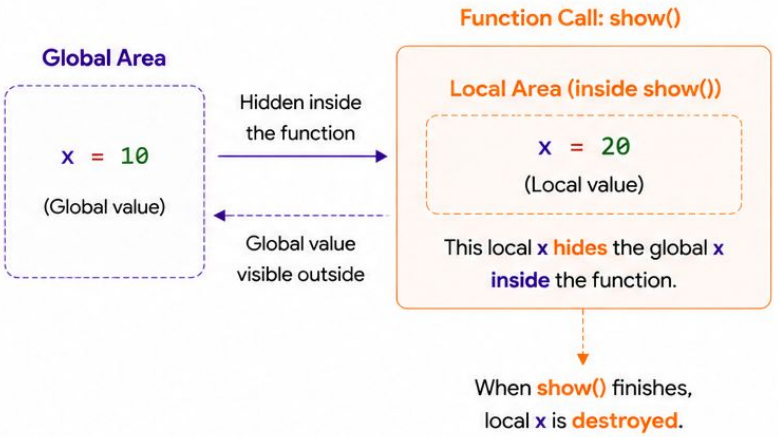
```
x = 10      # Global variable
def show():
    x = 20      # Local variable (shadows global)
    print("Inside function, x =", x)

show()
print("Outside function, x =", x)
```

Output

```
Inside function, x = 20
Outside function, x = 10
```

How it Works



The global variable is **not changed**.
The local variable exists **only during** the function call.

Remember: Local comes first!
Inside a function, Python looks for local variables first.
If not found, it looks in the global scope.

© Copyright Intellipaat. All rights reserved.

Modifying Global Variables (Without Global)

If we assign a value to a variable inside a function **without** using **global**, Python creates a **new local variable**.

Example

```
x = 10       # Global variable

def update():
    x = 20     # This creates a LOCAL x
    print("Inside function, x =", x)

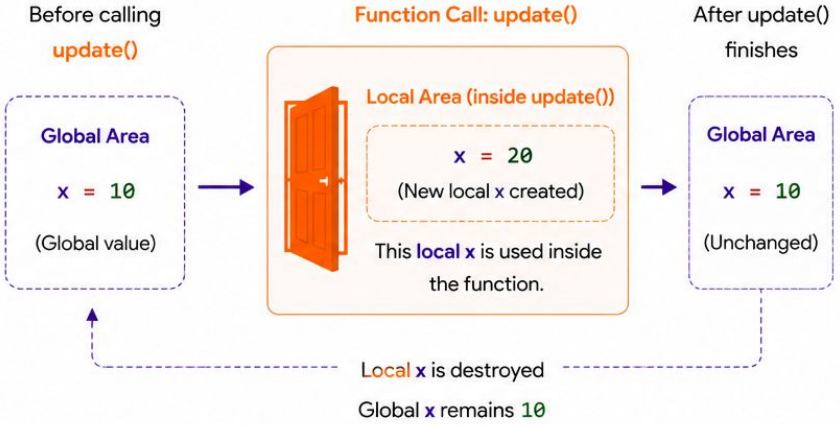
update()
print("Outside function, x =", x)
```

Output

```
Inside function, x = 20
Outside function, x = 10
```

Even though we assigned `x = 20` inside the function, the **global** `x` remains `10`.

How it Works



Without **global**, Python treats `x` inside the function as a separate **local** variable. Any changes affect only the **local** copy, not the **global** variable.

© Copyright Intellipaat. All rights reserved.

Modifying Global Variables (With Global)

To **modify** a global variable inside a function, we use the **global** keyword.
This tells Python to use the **existing global variable**.

Example

```
x = 10        # Global variable

def update():
    global x    # Use global to modify
    x = 20      # Modifies the global x
    print("Inside function, x =", x)

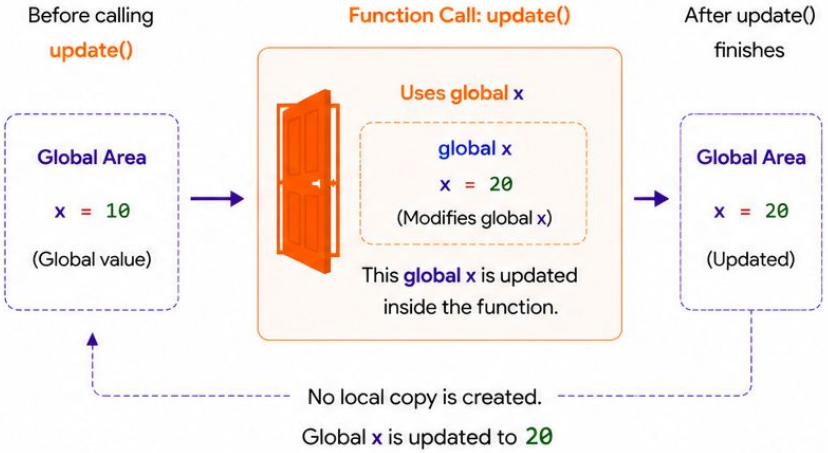
update()
print("Outside function, x =", x)
```

Output

```
Inside function, x = 20
Outside function, x = 20
```

Using **global** allows the function to access and modify the variable in the **global scope**.

How it Works



When we use **global x**, Python knows we want to work with the variable in the **global scope**, not create a new local one.

© Copyright Intellipaat. All rights reserved.

Variable Lookup Problem

When Python sees a variable name, how does it decide **which one to use?**

Example

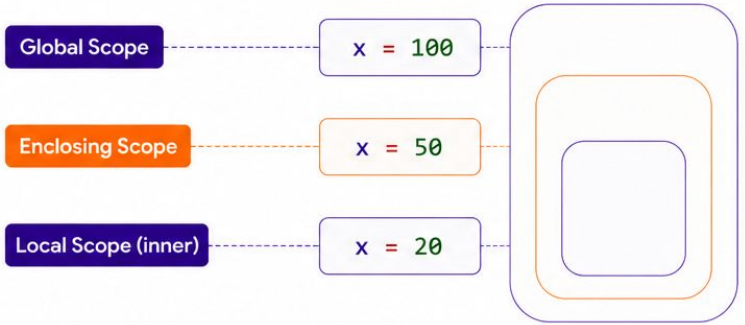
```
x = 100      # Global x
def outer():
    x = 50     # Enclosing x
    def inner():
        x = 20    # Local x
        print("Value of x:", x)
    inner()
outer()
```

What will this print?

A. 20
B. 50
C. 100
D. Error

What's Happening?

There are three variables with the same name **x**, but in different scopes.



Python follows a **specific rule** to find the right variable.
What do you think that rule might be?

Your Task: Predict the output and think about the order in which Python looks for variables.

© Copyright Intellipaat. All rights reserved.

The LEGB Rule

Python looks for a variable in a specific order (top to bottom) until it finds the first match.

What is LEGB?

LEGB is the order Python follows to resolve variable names.

- **L — Local**
  Inside the current function
- **E — Enclosing**
  In any enclosing (outer) function
- **G — Global**
  In the global area
- **B — Built-in**
  Python's built-in names (like `print`, `len`, `range`)

How Python Searches

Python searches from top to bottom.
The first match it finds, it uses!



1. **Local**
   Check inside the current function.
2. **Enclosing**
   Check in any enclosing functions.
3. **Global**
   Check in the global scope.
4. **Built-in**
   Check in Python's built-in names.

Example

If Python sees 'x', it checks here:

1. Local
2. Enclosing
3. Global
4. Built-in

Stops at the first match!

Think of LEGB as a search path. Python never looks below once it finds the answer!

© Copyright Intellipaat. All rights reserved.

LEGB Rule in Action

Let's trace how Python resolves the variable name **'x'**

Example

```
x = 100       # Global x
def outer():
    x = 50      # Enclosing x
    def inner():
        x = 20      # Local x
        print(x)    # Which x?
    inner()
outer()
```

Output

```
20
```

Python stops as soon as it finds a match. It **never** looks below that!

How Python Resolves 'x'

When `inner()` tries to `print(x)`, Python searches for 'x' using the **LEGB** rule.

L — **1. Local** (inner function)
Is 'x' defined inside `inner()`?
Yes! `x = 20` → Found!

E — **2. Enclosing** (outer function)
(Not checked because we already found it in Local) → Skipped

G — **3. Global** (module level)
(Not checked) → Skipped

B — **4. Built-in** (Python built-ins)
(Not checked) → Skipped

Result: Python found 'x' in the **Local** scope of `inner()` → `20` is printed.

Search Path (LEGB)

Python searches in this order:

- **L** — **Local** inner()
- **E** — **Enclosing** outer()
- **G** — **Global** (module level)
- **B** — **Built-in** (Python built-ins)

Top to bottom
First match wins!

Key Takeaway: Python uses the **LEGB** rule to find variables. It checks **Local** first, then **Enclosing**, then **Global**, and finally **Built-in**.

© Copyright Intellipaat. All rights reserved.

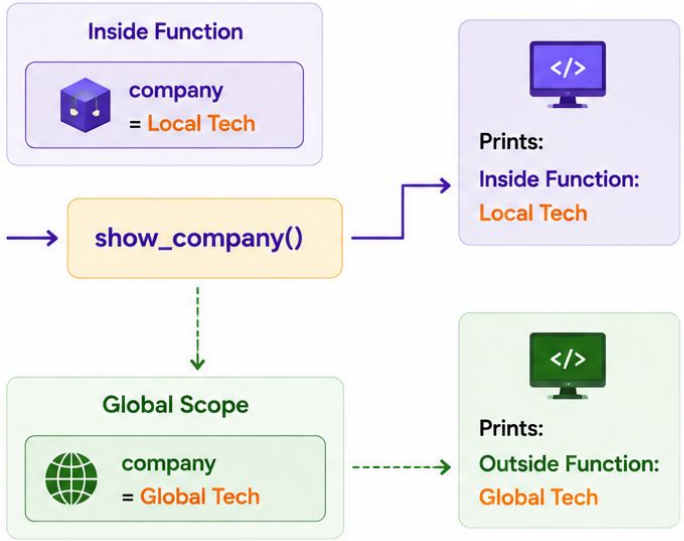
Hands-On: Scope

© Copyright Intellipaat. All rights reserved.

Hands-On: Scope

Problem Statement

- Create a global variable named company with the value Global Tech.
- Then create a function named show_company and define another variable with the same name, company, inside the function with the value Local Tech.
- Print the local value from inside the function.
- After calling the function, print the global value outside the function.
- This demonstrates that a local variable can use the same name as a global variable without changing the global value.



Expected Output

```
Inside Function: Local Tech
Outside Function: Global Tech
```

© Copyright Intellipaat. All rights reserved.

Type Hinting

© Copyright Intellipaat. All rights reserved.

What is Type Hinting?

Type Hints

Type hints let us specify the **expected data type** of variables, function parameters, and return values.

They don't change how Python runs the code, but they help humans (and tools) understand it better.

Not Using Type Hints

Hard to understand what is expected.

```
def greet(name):
    print(f"Hello, {name}!")
```

Call examples:

`greet("Hemant")`	Works
`greet(123)`	Works, but what is name expected to be?
`greet(None)`	Works, but what is name expected to be?

No clarity, more chances of wrong usage.

Using Type Hints

Clear about what is expected.

```
def greet(name: str) -> None:
    print(f"Hello, {name}!")
```

Call examples:

`greet("Hemant")`	Correct
`greet(123)`	Type checker will warn
`greet(None)`	Type checker will warn

Clear expectations, fewer mistakes.

Think of Type Hints Like Labels on Boxes

Correct Usage
(Matches the label)



Everyone knows what should go inside.

Incorrect Usage
(Doesn't match the label)



It may still work physically, but it's not what we expected.

Benefits of Type Hints

Catch errors early
IDEs and tools can detect type mismatches before you run the code.

Improve readability
Code becomes self-documenting. Anyone reading it knows what types to expect.

Better collaboration
Teams understand the code faster and work with more confidence.

What Do Type Hints Do?

They DO:
Help developers, IDEs, linters, and static type checkers.

They DON'T:
Enforce types at runtime. (Python will still run the code.)

Result:
Better guidance while coding, fewer bugs, cleaner code.

Example: Catching Mistakes Early

```
def add(a: int, b: int) -> int:
    return a + b
```

```
add(10, 20)
add("10", 20)
```

Type checker will warn

Remember: Good programmers don't write more code, they write better code.

© Copyright Intellipaat. All rights reserved.

Parameter Type Hints?

We can specify the **expected type** for each function parameter using a colon (:) followed by the type.
This helps others (and tools) know what kind of data the function expects.

Example

```
def add(a: int, b: int):
    return a + b
```

Parameter name: a
Expected type: `int`

Parameter name: b
Expected type: `int`

Here, **a** and **b** are expected to be **integers**.
Python will still run the code even if other types are passed.

Note

You can also use other types like `float`, `str`, `bool`, `list`, etc.
(We'll explore more types later!)

What this means

Helps readers **understand** what the function expects.

Helps **IDEs and tools** give better suggestions and catch mistakes early.

Quick Check

What does the function below expect for parameters **a** and **b**?

```
def add(a: int, b: int):
```

- A. Strings
- B. Integers
- C. Floats
- D. Booleans

© Copyright Intellipaat. All rights reserved.

Return Type Hints

We can specify the **expected type** of a function's return value using an **arrow (->)** followed by the type.
This makes our code clearer and helps others (and tools) know what the function returns.

Example

Parameter
The value passed to the function

Type hint
Expected type of the parameter

Arrow (->)
Separates the parameters from the return type

Return type hint
Expected type of the return value

```
def area(radius: float) -> float:
    return 3.14 * radius * radius
```

The **arrow (->)** separates the parameters from the return type.
It doesn't affect how the function runs.

What this means

The function **area** expects a **float** as input and returns a **float**.

Helps readers, **IDEs**, and tools understand what comes back from the function.

Example Call

```
result: float = area(5.0)
# result will be a float (78.5)
```

Output
78.5

Note: Common return types are `int`, `float`, `str`, `bool`, `list`, etc.
(We'll explore more types in later modules!)

© Copyright Intellipaat. All rights reserved.

Why Use Type Hints?

Type hints make code easier to read, understand, and work with.

Key Benefits

Better Readability
Makes it clear what types of data a function expects and returns.

Catch Errors Early
Helps find type-related mistakes during development.

Better Tool Support
IDEs and tools provide better suggestions, autocomplete, and refactoring.

Without vs With Type Hints

Without Type Hints

```
def add(a, b):
    return a + b
```

→ ✗ Harder to understand expected types.

With Type Hints

```
def add(a: int, b: int) -> int:
    return a + b
```

→ ✓ Clear and self-documenting.



Type hints are like **signposts** in your code.

They guide **readers** and **tools** without changing how Python executes your program.



Small Hints, Big Impact!

© Copyright Intellipaat. All rights reserved.

Hands-On: Type Hinting

© Copyright Intellipaat. All rights reserved.

Hands-On: Type Hinting

Problem Statement

- Create a function named **calculate_distance** that receives the speed of a vehicle and the travel time.
- Add appropriate **type hints**:
  - speed should be a **float**
  - travel time should be an **int**
  - the function should return a **float**
- Use a speed of **72.5** km/h and a travel time of **3** hours.
- Return the total distance traveled and print it with a **meaningful label**.

What This Function Does



Type Hints Used



Expected Output

```
Total Distance: 217.5 km
```

© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.